

LAB 7 – Assignment

Name: Krish Singh

Roll: AID25072

Class: AID-G sem 1

Program 1: Test Order

```
class Order {
    private int orderId;
    private String customerName;
    private double itemPrice;

    Order(int id, String name, double price) {
        this.orderId = id;
        this.customerName = name;
        this.itemPrice = price;
    }

    public String getCustomerName() {
        return customerName;
    }

    public int getOrderId() {
        return orderId;
    }

    public double getPrice() {
        return itemPrice;
    }

    public double calculateTotal() {
        return itemPrice + 200;
    }

    public void display() {
        System.out.print("Order ID: " + orderId);
        System.out.print("Customer: " + customerName);
        System.out.print("Total Bill: " + calculateTotal());
        System.out.print("\n");
    }
}

class PrimeOrder extends Order {
    PrimeOrder(int id, String name, double price) {
        super(id, name, price);
    }

    public double calculateTotal() {
        return getPrice();
    }
}
```

```

class InternationalOrder extends Order {
    InternationalOrder(int id,String name,double price){
        super(id,name,price);
    }
    public double calculateTotal() {
        return getPrice() + 1000 + 200;
    }
}
public class TestOrder{
    public static void main(String[] args){
        Order[] array = new Order[3];
        array[0]= new Order(101,"Ananya",2000);
        array[1]= new PrimeOrder(102,"Rahul",3000);
        array[2]= new InternationalOrder(103,"Meera",4000);
        for(int i=0;i<array.length;i++){
            array[i].display(); // this is where runtime polymorphism happens.
        }
    }
}

```

Output:

```

/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.l
Order ID: 101Customer: AnanyaTotal Bill: 2200.0
Order ID: 102Customer: RahulTotal Bill: 3000.0
Order ID: 103Customer: MeeraTotal Bill: 5200.0

Process finished with exit code 0

```

Analytical Questions:

1. Why does static method break runtime polymorphism?

Ans: Static method breaks runtime polymorphism because it belongs to the class, not the object. Runtime polymorphism works only when method call depends on the actual object in memory. But static method call depends on the reference type at compile time. Here the `array[i].calculateTotal()`; will not check the object type. Instead it will convert it to `Order.calculateTotal()`. So, every object uses the same method, so no overriding, no polymorphism.

2. What type of binding occurs in overridden methods?

Ans: Dynamic binding (Late Binding) occurs in overridden methods. This means JVM decides which method to execute during program execution based on object type. Here compiler only know obj is type Order, but at runtime JVM sees actual object is of PrimeOrder so it calls PrimeOrder.calculateTotal()

3. How does the code make sure abstraction is retained?

Ans: Abstraction is achieved by using base class reference and overriding methods so caller uses common interface while subclasses provide specific implementations. Here Main method does not care about Prime order logic or International order logic all it does is calls display function. Each subclass internally decides its own billing logic. So, Main program knows only “Order calculates bill” but subclasses decide how bill is calculated. This hides implementation hence, achieving abstraction.

Program 2: Smart Vehicle Toll System

```
class Vehicle {
    private String vehicleno;
    double basetoll=0;
    Vehicle(String vn){
        this.vehicleno=vn;
    }
    public String getVehicleno(){
        return vehicleno;
    }
    void display(){
        System.out.print("Vehicle no :"+vehicleno);
        System.out.print("Toll to be paid : "+calculatetoll());
        System.out.print("\n");
    }
    public double calculatetoll(){
        return basetoll;
    }
}
class car extends Vehicle{
    car(String vn){
        super(vn);
    }
}
```

```

    public double calculatetoll(){
        return basetoll+100;
    }
}
class truck extends Vehicle{
    truck(String vn){
        super(vn);
    }
    public double calculatetoll(){
        return basetoll+300;
    }
}
class electricCar extends Vehicle{
    electricCar(String vn){
        super(vn);
    }
    public double calculatetoll(){
        return basetoll+50;
    }
}
public class SmartVehicleTollSystem{
    public static void main(String[] args){
        Vehicle[] vh = new Vehicle[3];
        vh[0]=new car("KA01AB1234");
        vh[1]=new truck("KA05CD5678");
        vh[2]=new electricCar("KA09EF9999");
        for(int i=0;i<3;i++){
            vh[i].display();
        }
    }
}

```

Output:

```

/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.lo
Vehicle no :KA01AB1234Toll to be paid : 100.0
Vehicle no :KA05CD5678Toll to be paid : 300.0
Vehicle no :KA09EF9999Toll to be paid : 50.0

Process finished with exit code 0

```

Analytical Questions

1. In the Vehicle–Car example, overridden methods are resolved at runtime, but variables are resolved at compile time.

Demonstrate this using a small code snippet and explain the difference.

Ans: Overridden methods like `calculateToll()` are resolved at runtime because JVM decides which method to execute based on the actual object, either car, truck or electric car. This is dynamic binding. Variables like `basetoll` are resolved at compile time based on reference type `Vehicle`. Variables do not participate in runtime polymorphism and are bound using static binding.

2. What happens if `calculateToll()` is static?

Ans: if `calculateToll()` is declared as static, this becomes a class method, not a object method. It will be resolved at compile time using the reference type `Vehicle`. The JVM will not check the actual object type, so runtime polymorphism will not occur. Instead of overriding, method hiding happens, and all objects will use the base class method.

Program 3: Smart University Course Fee System

```
class Program{
    private String name;
    private double fees;
    Program(String n, double fs){
        this.name=n;
        this.fees=fs;
    }
    public String getname(){
        return name;
    }
    public double getfees(){
        return fees;
    }
    public double calculatefees(){
        return fees;
    }
    public void display(){
        System.out.print("Student: "+name);
        System.out.print("Total Fee: "+calculatefees());
        System.out.print("\n");
    }
}

class Undergraduate extends Program{
    Undergraduate(String n, double fs){
        super(n,fs);
    }
    public double calculatefees(){
        return getfees()+5000;
    }
}

class Postgraduate extends Program{
    Postgraduate(String n, double fs){
        super(n,fs);
    }
    public double calculatefees(){
        return getfees()+10000;
    }
}

class PhD extends Program{
    PhD(String n,double fs){
        super(n,fs);
    }
    public double calculatefees(){
        return getfees()-8000;
    }
}

public class SmartUniversityCourseFeeSystem{
    public static void main(String[] args){
        Program[] p = new Program[3];
        p[0]=new Undergraduate("Arjun",50000);
        p[1]=new Postgraduate("Sneha",60000);
```

```
p[2]=new PhD("Vikram",70000);  
for(int i=0;i<3;i++){  
    p[i].display();  
}  
  
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.lo  
Student: ArjunTotal Fee: 55000.0  
Student: SnehaTotal Fee: 70000.0  
Student: VikramTotal Fee: 62000.0  
  
Process finished with exit code 0
```

Analytical Questions

1. Why must method signatures match in overriding?

Ans: For method overriding to occur, the method in the subclass must have the same method name, same parameters, and same return type as the method in the base class. If the signature changes, it becomes method overloading, not overriding. If parameters or return type differ, runtime polymorphism will not occur because the JVM will treat it as a different method. That's why matching method signature ensures proper overriding and enables dynamic binding.

2. Can constructors be overridden?

Ans: No, constructors can't be overridden. Overriding requires inheritance and dynamic binding, but constructors are not inherited, are called when an object is created and are resolved at compile time. However, constructors can be chained using `super()` to call base class constructor. But this is constructor chaining not overriding.

Program 4: Smart Digital Payment Processing System

```
abstract class Payment{
    private String transactionID;
    private String Payer;
    private double FinalAmount;
    Payment(String id,String n,double x){
        this.transactionID=id;
        this.Payer=n;
        this.FinalAmount=x;
    }
    public double calculateFinal(){
        return FinalAmount;
    }
    public String getID(){
        return transactionID;
    }
    public String getName(){
        return Payer;
    }
    public double getAmount(){
        return FinalAmount;
    }
    void display(){
        System.out.println("Transaction ID: "+transactionID);
        System.out.println("Payer: "+Payer);
        System.out.println("Final Amount: "+calculateFinal());
    }
}

class CreditCard extends Payment{
    CreditCard(String id,String n,double x){
        super(id,n,x);
    }
    public double calculateFinal(){
        if(getAmount() > 50000){
            return(getAmount()+((0.02)*getAmount()+500);
        }
        else {
            return getAmount();
        }
    }
}

class UPI extends Payment{
    UPI(String id,String n,double x){
        super(id,n,x);
    }
    public double calculateFinal(){
        if(getAmount() > 100000){
            return getAmount()-(0.01*getAmount());
        }
        else{
            return getAmount();
        }
    }
}
```



```

    }
}
class Crypto extends Payment{
    Crypto(String id,String n,double x){
        super(id,n,x);
    }
    public double calculateFinal(){
        return getAmount()+(0.05*getAmount());
    }
}
}
public class SmartDigitalPaymentProcessingSystem{
    public static void main(String[] args){
        Payment[] emp = new Payment[3];
        emp[0]= new CreditCard("TXN101","Arun",60000);
        emp[1]= new UPI("TXN102","Megha",1200000);
        emp[2]= new Crypto("TXN103","Rohit",50000);

        for(int i=0;i<3;i++){
            emp[i].display();
        }
    }
}
}

```

Output:

```

/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Transaction ID: TXN101
Payer: Arun
Final Amount: 61700.0
Transaction ID: TXN102
Payer: Megha
Final Amount: 1188000.0
Transaction ID: TXN103
Payer: Rohit
Final Amount: 52500.0

Process finished with exit code 0

```

Analytical Questions

1. Why must Payment be abstract?

Ans: The Payment class must be abstracted because it represents a general concept (blueprint), not a specific payment type. There is no meaning in creating a direct object as each payment type (Credit card, UPI, Crypto) calculates final amount differently. Declaring the base class as abstract prevents direct object creation, forces subclass to implement `calculateFinalAmount()` and ensures proper runtime polymorphism. That's how abstraction enforces design correctness.

2. Can abstract classes have constructors?

Ans: Yes, abstract classes can have constructors. Although we cannot create objects of an abstract class directly, its constructor is executed when a subclass object is created. When a subclass object is created, payment constructor executes first via `super()`, then the subclass constructor executes. This ensures proper initialization of common fields like transaction Id and payer name.

3. What happens if overridden method is static?

Ans: If `calculateFinalAmount()` is declared as static, it will be resolved using the reference type, Method hiding will occur instead of overriding, Dynamic binding will not happen and runtime polymorphism will be lost. So even if the array stores different payment objects, all will call the base class method. Therefore static methods cannot participate in runtime polymorphism.

Program 5: Smart Library Fine Management System

```
class LibraryMember{
    private int memberId;
    private String memberName;
    private int numberOfDaysLate;
    LibraryMember(int id,String name,int days){
        memberId=id;
        memberName=name;
        numberOfDaysLate=days;
    }
    public int getDaysLate(){
        return numberOfDaysLate;
    }
    public int getMemberId(){
        return memberId;
    }
    public String getMemberName(){
        return memberName;
    }
    public double calculateFine(){
        return numberOfDaysLate;
    }
    public void displayFine(){
        System.out.println("Member ID: "+memberId);
        System.out.println("Name: "+memberName);
        System.out.println("Fine: "+calculateFine());
    }
}

class Student extends LibraryMember{
    Student(int id,String name,int days){
        super(id,name,days);
    }
    public double calculateFine(){
        if(getDaysLate()>30){
            return 5*getDaysLate()+200;
        }
        else{
            return 5*getDaysLate();
        }
    }
}

class Faculty extends LibraryMember{
    Faculty(int id,String name,int days){
        super(id,name,days);
    }
    public double calculateFine(){
        return 2*getDaysLate();
    }
}

class Guest extends LibraryMember{
    Guest(int id,String name,int days){
        super(id,name,days);
    }
}
```

```

    }
    public double calculateFine(){
        if(getDaysLate()>15){
            return 10*getDaysLate()+500;
        }
        else {
            return getDaysLate()*10;
        }
    }
}
public class SmartLibraryFineManagementSystem{
    public static void main(String[] args){
        LibraryMember[] memb = new LibraryMember[3];
        memb[0]=new Student(1,"Arjun",40);
        memb[1]=new Faculty(2,"Meera",20);
        memb[2]=new Guest(3,"Rahul",20);
        for(int i=0;i<3;i++){
            memb[i].displayFine();
        }
    }
}

```

Output:

```

/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Member ID: 1
Name: Arjun
Fine: 400.0
Member ID: 2
Name: Meera
Fine: 40.0
Member ID: 3
Name: Rahul
Fine: 700.0

Process finished with exit code 0

```

Analytical Questions

1. Why does static method prevent dynamic binding?

Ans: Static methods belong to class and not the object. Dynamic binding requires JVM to decide method execution based on the actual object at runtime. If calculateFine() is static, it will be resolved at compile time, the reference type

determines the method call, subclass specific fine calculation will not execute. Hence, runtime polymorphism will not occur.

2. What is constructor shadowing?

Ans: Constructor shadowing occurs when constructor parameters have same name as the instance variables. Example for bugged base class:

```
public LibraryMember(int id, String name, int days) {  
    memberId = id;  
    name = memberName;  
    numberOfDaysLate = days;  
}
```

Here, name=memberName; is incorrect because, parameter name shadows the instance variable memberName and Assignment direction is wrong. Correct measure would be:

```
this.memberName=name;
```

Using this removes shadowing and ensures proper initialization.

3. Can variables be overridden in Java?

Ans: No variables cannot be overridden in java. They are resolved using compile time binding based on the reference type. If a subclass declares a variable with the same name, it hides the parent variable. This is called variable hiding, not overriding. Only methods can be overridden and participate in runtime polymorphism.

4. Where does runtime polymorphism occur?

Ans: Runtime polymorphism occurs when a base class reference refers to a subclass object, a method is overridden in the subclass, the method is called using the base reference. The JVM decides at runtime which subclass method to execute. Thus, runtime polymorphism occurs during method invocation through base class reference pointing to subclass object.

Program 6: Appendix : Employee Bonus System

```
public class Employeep{
    private String name;
    protected double salary;
    Employeep(String n,double s){
        this.name=n;
        this.salary=s;
    }
    void calculateSalary(){
        System.out.println("Employee Salary:"+salary);
    }
    public double bonus(float percent,int hours){
        return(percent/100.0)*salary;
    }
}

class Managerp extends Employeep{
    Managerp(String n, double s){
        super(n,s);
    }
    public double bonus(float percent, int hours){
        return 1.5*super.bonus(percent,hours);
    }
}

class Developerp extends Employeep{
    Developerp(String n,double s){
        super(n,s);
    }
    public double bonus(float percent, int hours){
        double overtimePay=500*hours;
        return super.bonus(percent,hours)+overtimePay;
    }
}

public static void main(String[] args){
    Employeep[] emparray= new Employeep[3];
    emparray[0] = new Employeep("Ravi",10000);
    emparray[1] = new Managerp("Anita",150000);
    emparray[2] = new Developerp("Kiran",40000);
    for(int i=0;i<emparray.length;i++){
        emparray[i].calculateSalary();
        System.out.println("Bonus: "+emparray[i].bonus(5.0f,8));
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/ho
Employee Salary:10000.0
Bonus: 500.0
Employee Salary:150000.0
Bonus: 11250.0
Employee Salary:40000.0
Bonus: 6000.0

Process finished with exit code 0
```

Sample Question

1. Identify the type of inheritance used.

Ans: Hierarchical inheritance is being used here because Managerp and Developerp both inherit from Employeep and one parent class has multiple child classes.

2. Which method demonstrates runtime polymorphism?

Ans: The method: bonus(float percent, int hours) displays runtime polymorphism because it is overridden in Managerp and Developerp and it is called using base class reference: emparray[i].bonus(5.0f,8);

JVM decides at runtime which subclass version to execute.

3. What is the role of super.bonus() in Managerp and Developerp?

Ans: super.bonus() calls the base class implementation of the bonus calculation. In Managerp multiple base bonus by 1.5 , In Developerp overtime pay to base bonus is added. It allows reuse of base logic while extending functionality.

4. Why is salary declared as protected and not private?

Ans: Salary is declared as protected and not private so that the subclasses can directly access it. If declared private, subclasses would not be able to access it directly.

Protected provides controlled accessibility within inheritance hierarchy.

5. What will happen if bonus() is declared as static in base class?

Ans: If bonus() is declared as static in base class, Method hiding occurs, compile time binding happens, runtime polymorphism is lost and subclass specific bonus logic will not execute. All calls will resolve to Employee.bonus()